

PricePilot 三人工种分工说明书

答辩用 · 基于实际代码核对 (提交 94f9d01 · 26 文件 · +4182 / -404)
三块之间只有两处交接点，各自可独立讲清、独立被追问

0 先分清「推荐」这个词的两种含义

本项目里有 两件完全不同的事都叫「推荐」，答辩时最容易互相打脸，必须先分开：

名称	代码位置	触发时机	归属
对话内排序推荐 (图里那个节点)	<code>recommend_node.py</code> (41) <code>rank_top3.py</code> (32)	用户发话 → 检索到商品 → 贝叶斯排序取 Top3	甲 (你)
首页按画像推荐 (这次要分出去的)	<code>recommend_service.py</code> (417) <code>recommend_router/</code> <code>HomeRecs.vue</code>	打开/刷新空白页 → 读画像 → 直接拉商品	待指派 (建议归乙)

必须记住的区别

前者是图内一个节点，输入是本次检索结果，产出 `ranked_top` 并推 `products` 帧；

后者是图外的独立 HTTP 服务，压根不进 LangGraph，输入是 Redis 里的用户画像与收藏夹，自己再调一次 Shopify。

1 三块划分总览

切分原则：按职责边界切，不把任何单个模块劈成两半。保证每个人讲自己那块时前后闭合，被追问不会牵出别人的未讲内容。

代号	负责范围	一句话定位	在流程中的位置
甲	意图分流 + 检索链节点 + Schema / 适配层	用户这句话该闲聊、该追问还是该检索；检索回来怎么变成统一商品对象、排序、输出	主链路主干 (已定，答辩原内容)
乙	用户状态与决策前记忆 + 首页画像推荐	「这个用户是谁、之前说过什么」如何进入决策，以及打开页面时按画像推什么	进图之前注入 + 图外独立接口
丙	算价 · 证据 · 对比报告 + 会话/用户 · 前端	价格怎么算准、结论凭什么可信、数据怎么落库、界面怎么交付	主链路之后与对外交付

交接点 (全场只有两处)

交接	交出去的东西	谁讲、谁只说一句
乙 → 甲	<code>shopping_graph.chat()</code> 进图前组装好 <code>memory_block</code> ，写进 <code>state</code> ；甲的两个节点 (<code>chat_node</code> / <code>output_node</code>) 直接复用	乙讲记忆怎么组装；甲只讲「拿到现成文本，末尾加一句『记忆是背景，冲突时以本次需求为准』」
甲 → 丙	甲产出 <code>Product</code> 对象列表与 <code>ranked_top</code> ；丙在其上算价、取证据、写报告	甲讲 Schema 契约；丙讲「吃到的是已经归一化的对象，所以不必关心 Shopify 原始字段」

2 甲 —— 意图分流与检索链节点（你的部分，已定）

内容	文件	关键设计点
意图分流三支 + 空查询转追问	intent_node.py (149) clarify_node.py shopping_graph.py:36 route_after_intent	chat / clarify / shopify_search 四选一；search 但检索词为空 → 退回追问
意图识别与输出归一化	intent_schema.py (14) price_pref 归一化、 compose_search_keyword	把 cheap / 便宜 / low 收敛成统一取值，避免拼写差异导致下游漏判
Shopify 检索 (MCP)	shopify_search_node.py (68) search_shopify.py (142)	MCP 调用 + 第二道空查询闸 + 检索失败降级为空候选
Schema 与适配层	shopping_schema.py (31) adapter.py (178)	state → 统一输入 → MCP 参数 → 统一 Product 对象
排序与输出节点	recommend_node.py (41) rank_top3.py (32) output_node.py (97)	贝叶斯加权取 Top3；注意：取证与人民币补齐也在这个节点内（与丙相邻）
闲聊分支	chat_node.py (58)	元问题/寒暄不走检索，但仍注入记忆与最近对话

你答辩时的核心论点

让「用户这句话到底该怎么处理」变成一个有守卫的确定性问题，而不是把原始输入直接丢给检索。

必须准备的三个细节（有实测数据支撑，最能加分）

- 为什么必须拦住空查询：**Shopify catalog 是宽松语义检索，任何 query 都返回 50 条。实测 '鸣潮' → 返回一味中药 *Rhizoma*；'来一个1000以内的鸣潮周边' → 返回一本讲鸡的书 *What the Chicken Knows*。所以空/无意义检索词等于随机抽 50 个商品给模型推荐。
- 同一道闸为什么设两处：**route_after_intent 拦在路由层，shopify_search_node 拦在发请求前。前者保证不「假装要检索」，后者保证不「真的发出去」——纵深防御，且第二处把降级暴露在日志里而不是伪装成「没货」。
- 预算语义的关键修正：**BUDGET_MIN_RATIO 原为 0.1，把「1000 以内」误解成 900~1000 区间；实测 50 条候选全被判「低于下限」丢弃 → 候选 0 条 → 前端显示空白。改为 0 后只保上限，与 API 侧口径一致。

3 乙 —— 用户状态与决策前记忆 + 首页画像推荐

内容	文件	关键设计点
四层记忆 · 存储层	<code>save/window_memory.py</code> (49) <code>save/summary_memory.py</code> (55) <code>save/long_memory.py</code> (102) <code>save/profile_memory.py</code> (36)	L1 Redis List (窗口) · L2 PostgreSQL (摘要 UPSERT) · L3 Chroma 向量 (长期偏好) · L4 Redis Hash (结构化画像)
四层记忆 · 提取层	<code>retrieval/SummaryAgent.py</code> (73) <code>retrieval/long_agent.py</code> (77) <code>retrieval/profile_agent.py</code> (88)	三个 Agent 各管一层：本次聊了什么 / 一贯偏好 / 结构性事实
记忆调度与组装	<code>memory/manager/</code> 三个文件 <code>memory/__init__.py</code> (47)	按 窗口 → 摘要 → 长期 → 画像 顺序拼成一段文本，由近到远，冲突时模型应更信靠前的
进图前注入 / 出图后回写	<code>shopping_graph._build_memory_block</code> <code>_last_answer</code> / <code>_finish_turn</code>	注入失败只降级成「本轮无记忆」；回写丢后台 task，不阻塞 SSE 的 done 帧
首页按画像推荐	<code>recommend_service.py</code> (417) <code>recommend_router/</code> <code>HomeRecs.vue</code> (194)	画像 → 收藏 → 示例 三级降级 ；指纹缓存（画像变了自动失效）； <code>refresh=1</code> 换一批
画像与推荐的口径对接	<code>routers</code> 读 <code>profile:{user_id}</code>	读 <code>frequent_categories</code> / <code>preferred_brands</code> 决定推什么； <code>budget_habit</code> 解析成预算； <code>avoid</code> + <code>disliked_brands</code> 组成黑名单

乙答辩时的核心论点

模型每一轮都必须重新知道「这个用户是谁」。记忆不是把历史塞进 prompt，而是**四层不同时效、不同结构的信息，各有分工、各有降级**。

必须准备的关键细节

- Checkpoint 与四层记忆的分工（高频追问点）**：`InMemorySaver` 存的是**图内运行状态**，进程内、重启即失效；四层记忆是**跨会话要记住的用户信息**，落 Redis / PG / 向量库。两者职责不同，不能互相替代。
- 画像为空时推荐为什么不是空白**：`MODE_PROFILE` 要求画像里至少有 `frequent_categories` 或 `preferred_brands`，否则降级到 `MODE_FAVORITES`（用收藏标题找同类），都没有才 `MODE_SAMPLES`（给 4 个示例问题、不给商品卡）。
- 缓存怎么保证不给人过期画像**：缓存 key 里带画像指纹（`profile_fingerprint`），画像一变指纹就变，命中失效自动重查——不需要手写失效逻辑。
- L3 为什么必须换成本地中文嵌入**：Chroma 默认的 `all-MiniLM-L6-v2` 是英文模型，对「只买黑色」「通勤戴」这类中文偏好的语义区分度差，且首次使用要联网下载 79MB。改为读 `EMBEDDING_LOCAL_PATH` 指向本地 `bge-base-zh`。

4 丙 —— 算价 · 证据 · 对比报告 · 会话与交付

内容	文件	关键设计点
价格引擎与优惠计算	<code>price_calculator.py</code> (178) <code>price_schema.py</code> (167) <code>currency.py</code> (170) <code>offer_builder.py</code> (105)	券互斥组 / 门槛 / 有效期 / 返现上限 / 证据 ID 合并去重; 金额全程 Decimal
商品标准化与同款识别	<code>sku_matcher.py</code> (92)	不同来源归一成同款再比价
证据体系与可信度	<code>review_fetch_tool.py</code> (187) <code>review_evidence.py</code> (200) <code>review_node.py</code> (242)	抓评论 → 清洗 → 引用不到的结论直接丢弃
对比与报告子图	<code>comparison_graph.py</code> (140) <code>compare_node.py</code> (337) <code>reporter_node.py</code> (255)	预处理 → ... → 有证据的对比报告; 独立子图
联网搜索商品评价	<code>web_search.py</code> (64)	为 Top3 补第三方证据 (与甲的 <code>recommend_node</code> 相邻)
会话 / 用户 / SSE	<code>session_repository.py</code> (210) <code>session_router/ · deps.py</code> (30) <code>chat_router.py</code> (94) <code>login_router/</code> (315)	三种 SSE 帧; 会话归属校验不一致返回 403; 软删除; 懒创建会话
前端	<code>App.vue</code> · <code>SessionSidebar.vue</code> <code>HomeRecs.vue</code> · <code>style.css</code>	会话侧栏 / 商品卡 (序号、证据、评分) / 收藏抽屉含价格历史

丙答辩时的核心论点

不能让模型编造价格理由。 价格是一个有约束的合同对象、不是 float; 每条结论都要能溯源到证据, 溯源不到就在生成阶段拦掉。

必须准备的关键细节

- 为什么价格要建成合同对象:** 一次付款同时受券互斥组、券门槛、有效期、返现上限、币种约束。测试用例明确覆盖: 负价/零价在合同层就拒绝、三位小数被拒绝、返现超过付款金额要报错、优惠大于商品金额被排除、`now` 必须带时区、有效期取选中券的最早截止。
- 「解释里查无出处的金额」怎么抓:** 报告中出现的金额必须能对应到证据 ID, 对不上的会被判定为模型编造并揪出; 同时纯统计数字 (条数、比例) 不会被误判成金额。
- 避雷项为什么是硬排除而不是降权:** 用户说「不要二手」是硬条件, 降权意味着可能仍然推给他。且只做字面匹配、不命中近义词——宁可漏排也不误排。
- 贝叶斯平均为什么必要:** $score = (v/(v+m)) \cdot R + (m/(v+m)) \cdot C$, $m=50$, 同 IMDB Top 250 口径。解决「5.0 分只有 3 条评价」压过「4.6 分有 656 条」; 无评价商品 `score=None` 并标注「暂无评价, 无法评估」, 而不是当 0 分。
- SSE 三种帧:** `text` (增量文本) / `products` (商品卡数据) / `done` (结束, 带 `saved=false` 表示答完但没存进历史)。断流有 `stream_broken` 错误码。
- 跨平台坑 (最能体现动手能力):** Windows 上 `uvicorn` 硬编码返回 `ProactorEventLoop`, 而 `psycpg` 异步模式要求 `SelectorEventLoop`, 必然报错; 最后改成同步驱动 + `asyncio.to_thread`。另有 `passlib 1.7.4` 与 `bcrypt ≥ 4.1` 不兼容 (初始化即抛错且被 `service` 层吞掉, 表现为「注册失败」), 改为直接调 `bcrypt`。

5 工作量与讲时对照

维度	甲	乙	丙
主要文件数	约 10	约 15	约 20
主要代码行	约 700	约 1100	约 2100
自动化测试	意图路由 20 条	记忆端到端实测	价格 26 + 对比报告 38 + 会话/推荐 54
建议讲时	7-8 分钟	9-10 分钟	12-14 分钟
演示重点	一句模糊需求 → 是否被追问	刷新首页 → 画像变了推荐跟着变	一张对比报告 → 证据可溯源

如果丙嫌偏多

可把「前端（App.vue / SessionSidebar.vue / HomeRecs.vue / style.css）」整体移给乙。理由是首页推荐卡本身就是乙的「首页按画像推荐」的展示层，两人接口一致（/api/recommendations 返回的 mode + products），不会出现接口互相甩锅。移完后：乙约 1300 行、丙约 1900 行。

6 答辩前必须处理的三个风险

风险	具体问题	处理建议
记忆测试跑不起来	tests/test_memory.py (404) 仍 import 旧路径 app.ai.memory.*，而模块已迁到 app.ai.agent.memory.*；且旧测试全同步 + 注入 stub 模型，新实现全 async 且模型由单例自取。现场一跑即 ModuleNotFoundError。	三选一：① 按新 API 重写（功能已实测跑通，重写后能真跑）；② 表述改为「记忆走端到端实测」；③ 直接删除该失效文件
「推荐」一词混淆	甲讲「我做了推荐与输出」，乙讲「我做了首页推荐」，评委极可能问「你俩做的是不是一件事」。	开场各用一句话界定：甲讲 对话内的排序节点 ，乙讲 打开页面时按画像推 ，并主动说明二者数据来源不同（见第 0 节）
取证职责相邻	recommend_node（甲）内同时做了 Top3 取证与人民币补齐，而取证工具 web_search.py、汇率 currency.py 归丙。	甲只讲「排序取 Top3 后并发取证，失败降级为空证据」；丙讲「取证与汇率工具本身、以及报告级证据体系」。被追问细节时明确交棒

7 建议的答辩顺序

1. 甲先讲 2 分钟总览：整条链路是怎么串的（意图 → 检索 → 排序 → 输出），让评委先有全图。
2. 乙讲用户状态：承接甲，说明「进图之前」发生了什么。交界句：「甲的两个节点拿到的记忆块是现成文本，我这边负责它是怎么组装、失败怎么降级。」
3. 丙讲算价与证据：承接甲，说明「检索结果之后」发生了什么。交界句：「甲交给我的是已归一化的商品对象，所以我不用关心 Shopify 原始字段。」
4. 三人各留一个可现场演示的点（见第 5 节表格末行），避免纯讲代码。

本文档基于仓库实际代码核对生成（提交 94f9d01「完善推荐系统」，26 文件 / +4182 -404）。所有文件行数为实测值，括号内数字即行数。

文中标注的实测数据（Shopify 宽松检索返回无关商品、BUDGET_MIN_RATIO 导致候选清零）均来自代码注释中记录的原始排查过程，可直接引用。